

TECHNICAL PROJECT PROPOSAL

IDEA SUBMISSION

FADER

Fatigue Awareness and Driver Engagement Routing

Domain	Transportation and Logistics
Problem Statement No.	Omni_Transport_9
Problem Statement	Reducing Fatigue Related Trucking Accidents
Team Name	bi.Variates
Team Members	1. Abhinav Bute 2. Atharva Kale

1. Problem Understanding

Driver fatigue is a leading contributor to fatal collisions on long-haul and regional transit corridors, where drivers face extended exposure to low-stimulus driving conditions. Unlike acute distraction, fatigue-related impairment develops gradually and is frequently unrecognized by the driver until reaction capacity has already degraded. FADER is motivated by three interlocking physiological and environmental failure modes that are especially pronounced on monotonous, long-duration regional routes.

1.1 Highway Hypnosis (Monotony-Induced Trance)

Prolonged exposure to a repetitive visual field — a straight road (for eg. Samurddhi Mahamarg), uniform lane markings, and unchanging scenery — reduces the volume and variability of sensory input reaching the reticular activating system. This produces a dissociative, trance-like state commonly termed **highway hypnosis**, in which a driver's eyes remain open and fixed on the road while executive attention disengages. Steering and speed control continue as an automated motor pattern, but hazard perception, situational awareness, and reaction time collapse well before any outward sign of drowsiness appears.

1.2 Cabin CO₂ Build-Up

In a sealed cabin with recirculated air — the default HVAC mode on most highway trips for fuel efficiency and noise reduction — carbon dioxide concentration rises steadily as the driver exhales into a fixed air volume. Cabin CO₂ has been shown to climb from ambient levels near 400–500 ppm to over 2,500–3,000 ppm within 30–60 minutes of recirculated operation with a single occupant. Concentrations above roughly 1,000 ppm are associated with measurable declines in cognitive performance, alertness, and decision-making speed; above 2,500 ppm, drowsiness and headache onset accelerate sharply. This is a silent, self-reinforcing hazard: the fatigue it induces increases the driver's motivation to keep the cabin sealed and quiet, which further elevates CO₂.

1.3 Microsleeps

A microsleep is an involuntary lapse into sleep lasting a fraction of a second to several seconds, during which the driver's eyes may close partially or fully and voluntary muscle control is briefly suspended. Microsleeps occur without conscious awareness — drivers routinely underestimate both their frequency

and duration — and at highway speeds, even a two-second lapse translates to dozens of meters of uncontrolled travel. Microsleeps are the terminal symptom of the fatigue chain: they are preceded by measurable changes in blink rate, eyelid aperture (PERCLOS), and gaze fixation duration, all of which are detectable through facial tracking before the lapse itself occurs.

1.4 Why Monotonous Regional Transit Amplifies the Risk

- **Route homogeneity:** Long stretches of regional highway offer little cognitive stimulus, accelerating the onset of highway hypnosis relative to urban driving.
- **Extended single-occupancy segments:** Regional transit often lacks a co-pilot or conversational partner, removing a natural fatigue countermeasure and allowing CO₂ to accumulate unchecked.
- **Time-of-day exposure:** Regional schedules frequently span circadian low points (late night, post-lunch), compounding fatigue with the body's natural alertness dips.
- **Delayed self-recognition:** Because highway hypnosis and CO₂-driven drowsiness both suppress the very metacognitive awareness needed to notice them, drivers systematically underestimate their own impairment until a microsleep — or a near-miss — occurs.

Core Insight

Each cause above is individually well-studied, but they compound: monotony induces hypnosis, hypnosis reduces the impulse to ventilate the cabin, rising CO₂ deepens drowsiness, and drowsiness culminates in microsleeps. FADER is designed to interrupt this chain at every link rather than reacting only to the final symptom.

2. Proposed Solution

FADER (Fatigue Awareness and Driver Engagement Routing) is proposed as a **proactive digital co-pilot** — a mobile-first IoT application that treats driver fatigue as a preventable, multi-signal engineering problem rather than a post-hoc alarm system. Where conventional drowsiness detectors are purely reactive (alerting only after eyelid closure is already detected), FADER combines three complementary intervention layers to keep the driver ahead of the fatigue curve.

2.1 Environmental Forecasting

FADER ingests route, weather, and time-of-day telemetry ahead of and during the trip to anticipate conditions known to accelerate fatigue — long monotonous stretches, night driving, and adverse weather — and pre-emptively adjusts its intervention intensity before symptoms of drowsiness appear.

2.2 Continuous Facial Tracking

Using on-device computer vision, FADER continuously monitors eyelid aperture, blink rate, and gaze direction as a real-time physiological ground truth for driver alertness, enabling early detection of the precursors to microsleep rather than the microsleep itself.

2.3 Conversational AI Engagement

Rather than relying solely on alarms, FADER uses a location- and context-aware conversational AI layer to actively re-engage the driver's cognitive attention — asking questions, offering trivia tied to the route, and modulating tone based on the detected fatigue state — restoring the mental stimulation that monotonous routes remove.

2.4 Design Philosophy

- **Proactive over reactive:** Intervene at the earliest detectable signal (rising CO₂, early PERCLOS drift) rather than waiting for a hard drowsiness threshold.
 - **Layered defence:** Environmental forecasting, biometric sensing, and conversational engagement operate concurrently so that failure or noise in one channel does not compromise the system.
 - **Non-intrusive by default, escalating under risk:** Baseline engagement is ambient (subtle UI, light conversation); intervention escalates only as measured risk increases, culminating in the hard alarm described in Section 4.
 - **Edge-first privacy and latency:** All biometric processing (facial tracking) executes on-device, ensuring zero-latency alarm response and ensuring raw camera frames never leave the vehicle.
-

3. System Architecture

FADER is architected as a deliberate **Edge-to-Cloud split**, separating latency-critical, privacy-sensitive processing from compute- and data-intensive services that benefit from centralized infrastructure. This division is the central engineering decision of the system: safety-critical inference must never depend on network availability, while route intelligence and generative dialogue are better served by cloud-scale resources.

3.1 Edge Layer — On-Device Processing

Edge Layer (Mobile)	Detail
Responsibility	Real-time Face Mesh landmark tracking, PERCLOS / blink-rate computation, microsleep and drowsiness scoring, alarm triggering.
Execution location	Entirely on-device (driver's mobile phone), inside the Flutter application runtime.
Rationale	Facial biometric inference must resolve in single-digit milliseconds and must function with zero network dependency — a dropped cellular connection on a regional highway cannot be allowed to disable the alarm.
Data handling	Camera frames are processed and discarded locally; only derived alertness scores (not raw imagery) may optionally be relayed upstream for logging.

3.2 Cloud Layer — FastAPI Backend

Cloud Layer (Backend)	Detail
Responsibility	Route mapping and turn-by-turn navigation, live weather and cabin-environment telemetry, LLM-driven conversational generation, session/trip state management.

Cloud Layer (Backend)	Detail
Execution location	Python FastAPI backend, deployed as a horizontally scalable async service.
Rationale	Mapping, weather forecasting, and LLM inference are compute- and API-intensive workloads unsuited to mobile hardware, and benefit from centralized caching, rate-limiting, and provider abstraction.
Transport	Persistent WebSocket connections stream weather/telemetry updates and LLM dialogue turns to the client with low round-trip overhead, avoiding repeated REST handshake latency during an active trip.

3.3 Communication Flow

1. The Flutter client establishes a persistent **WebSocket** session with the FastAPI backend at trip start, authenticated via a short-lived trip token.
 2. The Edge layer runs MediaPipe Face Mesh locally at the device camera's native frame rate, computing a rolling alertness score with no round-trip to the server.
 3. The Cloud layer independently polls Mapbox for route/ETA updates and Tomorrow.io for weather telemetry, pushing incremental updates to the client over the same WebSocket channel.
 4. When the Edge layer's alertness score crosses an engagement threshold, the client emits a lightweight event over the WebSocket; the backend responds by invoking the LLM API to generate a contextual conversational prompt, streamed back to the client for text-to-speech playback.
 5. The **Dead Man's Switch** alarm path (Section 4.1) is intentionally kept out of this loop — it triggers entirely on-device so that alarm latency is never a function of network round-trip time.
-

4. Key Features

4.1 Three-Strike "Dead Man's Switch" Alarm

A tiered, escalating alarm protocol driven by the Edge layer's continuous facial tracking, designed to interrupt a fatigue episode before it becomes a microsleep — while avoiding alarm fatigue from over-triggering on brief, harmless eye closures.

- **Strike 1 (Soft alert):** Sustained eyelid closure or gaze drift beyond a calibrated PERCLOS threshold triggers a subtle audio chime and a gentle voice prompt from the conversational AI.
- **Strike 2 (Active intervention):** A second detection within a rolling window escalates to a louder alert, an explicit verbal check-in ("Are you still with me?"), and a suggestion to pull over or take a break.
- **Strike 3 (Hard alarm / Dead Man's Switch):** A third detection, or any single detection of a full microsleep-length closure, triggers a high-intensity audible and haptic alarm and flags the trip session — the on-device equivalent of an industrial dead man's switch, requiring active driver acknowledgment to reset.
- The strike counter decays over time during sustained normal alertness, preventing stale strikes from prematurely triggering the hard alarm on an otherwise attentive driver.

4.2 Predictive CO₂ Cabin-Flush Alerts

Rather than waiting for a cabin CO₂ sensor reading to cross a fixed threshold, FADER models the **rate of accumulation** based on trip duration, HVAC recirculation state, and occupancy, forecasting when the cabin will cross the cognitive-impairment threshold and prompting a ventilation ("cabin-flush") action before drowsiness sets in — turning a reactive sensor alert into a predictive one.

4.3 Location-Based LLM Trivia

The conversational AI layer uses the driver's live route position (via Mapbox) as grounding context for the LLM, generating short, engaging trivia or points-of-interest commentary about the region being traveled through. This re-engages cognitive attention through relevant, low-effort conversation rather than generic or repetitive prompts, directly counteracting the monotony that drives highway hypnosis.

4.4 Circadian Light Theme Enforcement at Night

FADER enforces a light-mode UI theme during nighttime driving hours, deliberately inverting the conventional "dark mode for night" assumption. A brighter interface increases short-wavelength light exposure to the driver's photoreceptors, supporting circadian alertness signalling, while paradoxically most dark-mode dashboards are optimized for reduced eye strain in stationary use rather than sustained alertness during a monotony-prone driving task. The theme switch is time- and ambient-light-aware, with brightness capped to avoid glare.

5. Technology Stack

The stack is selected to keep latency-critical inference on-device while delegating data-rich, API-dependent services to a lightweight, async-first backend.

Layer	Technology & Role
Frontend / Edge Runtime	Flutter — cross-platform mobile application shell, camera pipeline, and UI/UX layer, including circadian theme switching.
Backend API	Python / FastAPI — asynchronous REST and WebSocket service layer orchestrating mapping, weather, and LLM calls.
Facial Tracking	Google MediaPipe (Face Mesh) — on-device landmark detection for PERCLOS, blink rate, and gaze estimation.
Routing & Mapping	Mapbox — turn-by-turn navigation, route geometry, and location context for LLM grounding.
Weather / Environmental Telemetry	Tomorrow.io — real-time and forecast weather data feeding the environmental forecasting layer.
Conversational Engine	LLM API — context-aware dialogue generation for engagement prompts and location-based trivia.

6. Implementation Plan

Given the fixed submission deadline [01/09/2026], the implementation plan is structured as a rapid, parallelized prototyping sprint organized into four actionable phases rather than a calendar-based timeline. Phases are designed with partial overlap so that backend and edge workstreams can progress concurrently across the development team.

Phase A — Core Setup & Scaffolding

Establish the foundational skeleton of both the mobile client and backend service so all subsequent workstreams have a stable base to build against.

- Initialize the Flutter project shell: navigation structure, camera permission pipeline, and base UI theme (light/dark) scaffold.
- Stand up the FastAPI backend skeleton with a working WebSocket handshake and health-check endpoint.
- Integrate MediaPipe Face Mesh into the Flutter camera pipeline and confirm real-time landmark output on-device.

Phase B — Telemetry Integration

Connect the external data providers and establish the data pipelines feeding both the predictive and conversational features.

- Integrate Mapbox routing SDK for live route, ETA, and location-context data.
- Integrate Tomorrow.io weather API and build the environmental forecasting module.
- Implement the CO₂ accumulation model (trip duration, recirculation state, occupancy) driving predictive cabin-flush alerts.
- Establish the WebSocket telemetry stream from backend to client for route/weather push updates.

Phase C — Intelligence & Feature Integration

Layer the fatigue-detection logic and conversational AI on top of the telemetry and tracking foundations from Phases A and B.

- Implement PERCLOS/blink-rate scoring and the three-strike Dead Man's Switch state machine on the Edge layer.
- Wire the LLM API into the FastAPI backend for context-grounded conversational prompts and location-based trivia generation.
- Implement circadian light-theme enforcement logic tied to time-of-day and ambient light sensing.
- Connect alarm escalation events end-to-end: Edge detection → local alarm trigger → optional backend session logging.

Phase D — Integration Testing & Demo Hardening

Consolidate all workstreams into a single cohesive, demo-ready build ahead of the August 31st deadline.

- End-to-end integration testing across a simulated long-route driving session (route + weather + facial tracking + alarm + trivia).
- Latency validation of the Edge alarm path to confirm zero-dependency operation under simulated network loss.
- UI/UX polish pass, including day/night theme transition and alarm escalation visuals.
- Prepare demo script, recorded walkthrough, and Phase 1 submission materials.